

N-Metrics: The Exact Thermodynamics of Consensus

Technical Documentation

Marco N Framework

May 2, 2026

Contents

1	N-Metrics: The Exact Thermodynamics of Consensus	2
1.1	General Library Overview	2
1.2	Simulation Engine and Scenario Generator	2
1.2.1	What does the Scenario Generator do?	2
1.2.2	Coverage Analysis (Stress Testing)	3
1.3	Installation	3
1.4	Terminal Usage (CLI) - Quick Examples	3
1.5	Python API Usage	4

1 N-Metrics: The Exact Thermodynamics of Consensus

`nmetrics` is an advanced Python library designed to evaluate reliability and consensus in empirical data matrices. It proposes a new topological and thermodynamic framework (The N Framework) that overcomes the limitations of asymptotic frequentist inference, placing the population at the center of the Confidence Interval via IPW (Weighted Simulation).

In addition to the new Natural coefficients (NI , NN , NO), the library includes ultra-optimized engines (vectorized and in C) to calculate classical consensus estimators and their Classical Bootstrap resampling, as well as an advanced Monte Carlo simulation engine to subject estimators to stress tests.

1.1 General Library Overview

The architecture of `nmetrics` is divided into three fundamental topologies. Each features its Natural Coefficient, classical contrast estimators, and an arsenal of command-line interface (CLI) commands for empirical analysis and theoretical simulations.

Topology	Scale	Natural Coefficient	Classical Estimators	CLI Analysis Commands	Stress Test Commands
Interval	Continuous Numeric	NI (Natural Interval)	AKI (Krippendorff's Alpha) ICC(2,1) (F-ANOVA)	<code>nmetrics-ni</code> <code>nmetrics-aki</code> <code>nmetrics-icc21</code>	<code>nmetrics-ni-cob</code> <code>nmetrics-aki-cob</code> <code>nmetrics-icc21-cob</code>
Nominal	Categorical (Unordered)	NN (Natural Nominal)	AKN (Krippendorff's Alpha) KF (Fleiss' Kappa)	<code>nmetrics-nn</code> <code>nmetrics-akn</code> <code>nmetrics-kf</code>	<code>nmetrics-nn-cob</code> <code>nmetrics-akn-cob</code> <code>nmetrics-kf-cob</code>
Ordinal	Categorical (Ordered)	NO (Natural Ordinal)	AKO (Krippendorff's Alpha) W (Kendall's W)	<code>nmetrics-no</code> <code>nmetrics-ako</code> <code>nmetrics-w</code>	<code>nmetrics-no-cob</code> <code>nmetrics-ako-cob</code> <code>nmetrics-w-cob</code>

Table 1: Architecture and command matrix of the `nmetrics` library.

Note on Global and Comparative commands: In addition to individual commands, `nmetrics-interval`, `nmetrics-nominal`, and `nmetrics-ordinal` can be used for comparative empirical analyses. For massive stress simulations that pit the three engines against each other simultaneously, use `nmetrics-[family]-comp`.

1.2 Simulation Engine and Scenario Generator

The core of the library's mathematical validation resides in its `generator_escenarios` module. This engine allows recreating virtual rater panels (Subjects $\times$ Raters) under controlled conditions of thermodynamic noise and bias.

1.2.1 What does the Scenario Generator do?

It allows modeling human evaluation behavior by injecting “parametric noise” (for interval scales) or “categorical noise” (for nominal/ordinal scales). It includes 5 predefined universal scenarios:

1. **Almost Null (Casi Nulo):** Agreement destroyed by maximum entropy (extreme noise).
2. **Random (Aleatorio):** Responses generated by pure combinatorial chance.

3. **Reasonable (Razonable):** Typical human consensus with moderate local disagreements.
4. **Almost Perfect (Casi Perfecto):** Highly calibrated raters with minimal deviations.
5. **Almost Identical (Casi Idéntico):** Absolute unanimity or topological variance tending to zero.

1.2.2 Coverage Analysis (Stress Testing)

Thanks to the CLI commands ending in `-cob` and `-comp`, it is possible to cross-test the scenario generator against the estimators. These commands launch massive Monte Carlo experiments that:

- Generate dynamic matrices based on the described scenarios.
- Establish the **Thermodynamic Population Truth** (the exact topological center of the *N* Framework).
- Evaluate whether the generated Confidence Interval manages to cover that population truth, empirically revealing the biases of traditional estimators.

1.3 Installation

The library is installed in your virtual environment by executing the following command from the root of the project (where the `pyproject.toml` file is located):

```
pip install -e .
```

1.4 Terminal Usage (CLI) - Quick Examples

The installation grants access to 24 complete terminal commands.

1. Individual Empirical Analysis with Audit (N Framework)

Execute the Natural Ordinal Coefficient (*NO*), audit anomalies at 2.5 sigmas to detect problematic raters, and export the replicas:

```
nmetrics-no clinical_surveys.csv -u 2.5 --exportar output_replicas.csv
```

2. Global Comparative Table of Empirical Data

Analyze categorical data saved with commas (the library automatically translates isomorphisms like 'A', 'B' to numerical values):

```
nmetrics-nominal diagnoses.csv -k 7 -s "," -r 5000
```

3. Launch an Individual Mathematical Coverage Stress Test

Simulate 500 panels of 10 raters evaluating 100 subjects on a 5-point scale to audit *NI*:

```
nmetrics-ni-cob -n 100 -m 10 -k 5 -e 500 -o validation_ni.csv
```

4. Estimator Battle (Global Stress Test)

Pit the three ordinal metrics (*NO*, *AKO*, Kendall's *W*) against each other in a Monte Carlo simulation of 100 experiments:

```
nmetrics-ordinal-comp -n 50 -m 7 -k 5 -e 100 -r 1000 -o
    comparative_ordinal.csv
```

1.5 Python API Usage

For integrations into data science pipelines, *nmetrics* exposes a clean architecture of mathematical cores and simulators accessible via API.

```
import numpy as np
from nmetrics.interval import calcular_estadisticas_ni
from nmetrics.simulation.generador_escenarios import
    generar_matriz_dinamica

# 1. Simulate panel: 30 patients, 5 doctors, scale 1-10 (Scenario '
    Reasonable')
# Note: String parameters are kept in Spanish to match library
    source code
matriz_simulada = generar_matriz_dinamica(n_sujetos=30, m_jueces=5,
    k_escalas=10,
                                     escenario='Razonable',
                                     tipo_escalas='
                                     parametrica')

# 2. Execute Interval Weighted Simulation engine
ni_mu, ni_pob, ic_inf, ic_sup, *_ = calcular_estadisticas_ni(
    matriz_simulada,
    S_replicas=2000,
    k_escalas=10,
    metodo_ic='SP'
)

print(f"Empirical Agreement (Sample): {ni_mu:.4f}")
print(f"Population Agreement (Projected): {ni_pob:.4f}")
print(f"Panel Confidence Interval: [{ic_inf:.4f}, {ic_sup:.4f}]"
```

Notes for the Researcher:

- All inference engines tolerate **missing data (NaN)** natively without requiring prior imputation.
- In the **Ordinal** topology, the topological anomaly audit evaluates the columns (Raters), while in the **Interval** and **Nominal** families, it evaluates the rows (Subjects).
- The Natural Coefficients include a Level III that analytically calculates the exact Thermodynamic Chance Limit based on the combinatorics of the hyperspace, reporting the topological validity of the consensus through its **exact Topological p -value** (p_{top}).